

Архитектура проекта: Библиотека алгоритмов для графов

Общее описание:

Проект представляет собой библиотеку, реализующую шесть классических алгоритмов для решения задач поиска кратчайших путей в графах. Библиотека предназначена для использования в задачах анализа графов и исследований производительности алгоритмов.

Все компоненты библиотеки реализованы на языке C++ и упакованы в модульную архитектуру, поддерживающую расширяемость, читаемость и тестируемость.

Структура проекта:

Проект разделен на следующие ключевые компоненты:

1. Класс **Graph** — для представления структуры графа.
2. Класс **GraphAlgorithms** — интерфейс для использования алгоритмов.
3. Реализация алгоритмов — отдельные модули для каждого алгоритма.
4. Среда исполнения — для тестирования и измерения производительности.
5. Тесты — для проверки корректности реализации и производительности.

1. Класс **Graph** - структура графа, поддерживает работу с матрицей смежности и операцию получения соседей.

Файлы:

- `graph.h` — описание класса и реализация методов класса.

Поля:

- `bool isDirected` — флаг, указывающий, ориентированный граф или нет.
- `int vertexCount` — количество вершин в графе.
- `int edgeCount` — количество рёбер в графе.
- `std::vector<std::vector<int>> adjMatrix` — матрица смежности графа.

Методы:

- `bool isDirectedGraph() const` — метод возвращает флаг, указывающий, является ли граф ориентированным.
- `int getNumVertices() const` - возвращает количество вершин в графе.
- `int getNumEdges() const` - возвращает количество рёбер в графе.
- `const std::vector<std::vector<int>>& getAdjMatrix() const` - возвращает ссылку на матрицу смежности графа.
- `std::vector<std::pair<int, int>> getNeighbors(int vertex) const` - возвращает вектор пар (вершина-сосед, вес ребра).

2. Класс GraphAlgorithms

Файлы:

- graphAlgorithms.h — интерфейс для вызова алгоритмов.

Назначение: Класс предоставляет интерфейс для вызова шести алгоритмов. Каждый алгоритм реализован как отдельная функция, что делает библиотеку расширяемой и модульной.

Методы:

- bfs() — реализация алгоритма поиска в ширину (BFS).
- dijkstra() — реализация алгоритма Дейкстры.
- bellmanFord() — реализация алгоритма Беллмана-Форда.
- aStar() — реализация алгоритма A*.
- floydWarshall() — реализация алгоритма Флойда-Уоршалла.
- oneKBfs() — реализация алгоритма 1-k BFS.
- zeroOneBfs() — реализация алгоритма 0-1 BFS.
- bruteForce() — наивная реализация.

Каждый метод представлен в заголовке graphAlgorithms.h, а его реализация находится в соответствующем алгоритму .cpp файле.

3. Реализация алгоритмов

Каждый алгоритм реализован в отдельном модуле, включающем файл для реализации и тесты для проверки корректности и производительности.

- BFS: bfs.cpp, тесты — testBfs.cpp.
- Dijkstra: dijkstra.cpp, тесты — testDijkstra.cpp.
- Bellman-Ford: bellmanFord.cpp, тесты — testBellmanFord.cpp.
- A*: aStar.cpp, тесты — testAStar.cpp.
- Floyd-Warshall: floydWarshall.cpp, тесты — testFloydWarshall.cpp.
- 1-k-bfs: oneKBfs.cpp, тесты — testOneKBfs.cpp.
- 0-1-bfs: zeroOneBfs.cpp, тесты — testZeroOneBfs.cpp.
- Brute-Force: bruteForce.cpp, тесты — testBruteForce.cpp.

4. Среда исполнения - используется для тестирования и сравнения производительности алгоритмов.

- Измерение времени выполнения алгоритмов на графах разных типов и размеров.
- Генерация отчетов о производительности.
- Возможность настройки параметров графа (число вершин, плотность ребер, наличие отрицательных весов).

5. Тесты - проверяют корректность алгоритмов

- Тесты для алгоритмов: testBfs.cpp, testDijkstra.cpp и другие.

